BioEval: An LLM-Driven Framework for Evaluating Dataset Transparency, Reproducibility, and Information-Theoretic Rigor in Computational Simulation Papers

Joel Dietz (ORCID 0009-0007-4948-9192)

June 6, 2026

Abstract

Computational and agent-based simulation papers increasingly drive claims about biological coordination — ant and termite stigmergy, honeybee colony dynamics, firefly synchronization, and ant-colony optimization — yet their reproducibility is rarely assessed systematically. Two failures recur: datasets and code are declared without resolvable, verifiable provenance, and systems whose entire subject is information flow are described mechanistically without ever quantifying that information. BioEval is an LLM-driven evaluation framework that scores a paper on a versioned seven-dimension rubric. Six dimensions measure conformance to good data and reproducibility practice — data disclosure, dataset resolvability, code availability and versioning, code-to-data traceability, simulation derivation clarity, and reproducibility-package quality. A seventh, orthogonal dimension scores Information-Theoretic Rigor: whether the paper formalizes and quantifies the information content of the system it models (Shannon entropy, mutual and transfer entropy, channel capacity, communication bit rate). Scores are grounded in verified external evidence — resolved accessions and DOIs, live repository facts — rather than the paper's own claims. We apply BioEval (rubric v0.8.0) to a corpus of 14 insect-swarm and agent-based simulation projects. Transparency is uneven and reproducibility packaging is consistently weak; most strikingly, information-theoretic rigor is systematically low even though communication and coordination are the modeled phenomena. This deposition bundles the report together with the complete, MIT-licensed source code of the evaluation system.

Keywords: reproducibility · dataset transparency · information theory · agent-based simulation · stigmergy · swarm intelligence · code-to-data traceability · large language models · FAIR data

1 Introduction

Agent-based and computational simulations are now a primary vehicle for claims about biological coordination: how ant and termite colonies self-organize through stigmergy, how honeybee colonies allocate labor, how populations of fireflies synchronize, and how ant-colony optimization transfers these mechanisms to engineering. The credibility of such claims rests on two things that are easy to assert and hard to verify. First, the data and code behind a simulation must be disclosed in a way that a reader can actually resolve and re-run — declaring that a dataset 'is available' is not the same as providing a resolvable accession, a versioned repository, and a runnable package. Second, when the modeled system is itself a communication system, scientific rigor demands that the information it carries be formalized and quantified, not merely narrated.

BioEval addresses both gaps in a single instrument. It is an evaluation framework that submits a paper (by URL or PDF), extracts its real text, gathers external evidence, and uses a large language model to score the paper against a fixed, versioned rubric. The framework deliberately separates two orthogonal questions: does the work conform to good data and reproducibility practice, and — independently — does it bring information-theoretic rigor to a system whose very subject is information flow? This report explains how the system works and presents its evaluations of a corpus of insect-swarm and agent-based simulation projects.

2 The BioEval Framework

2.1 Pipeline

A submission is fetched through an SSRF-guarded path and its text is extracted from the real PDF (via a pure-JavaScript pdf.js build); if too little text can be recovered — for example a scanned, image-only document —

the evaluation errors out with a stated reason rather than scoring garbage. The extracted text is then analyzed by Anthropic's Claude in an evidence-gathering pass that resolves declared identifiers against Crossref and DataCite and inspects any linked code repository for live facts (license presence, releases, structure). Only after evidence is collected does the model assign the seven dimension scores together with a structured set of findings, gaps, and recommendations, all persisted and aggregated in a dashboard.

2.2 The seven dimensions

The rubric is versioned (currently v0.8.0, intentionally pre-1.0 while it is validated by reviewers) and stamped onto every evaluation so scores remain interpretable as the rubric evolves. Each dimension is scored 0–100 against fixed guideposts; the overall score is the weighted mean of all seven, using the weights below.

Dimension	Weight	What it tests
Data Disclosure	18%	Whether the paper names the data it uses and declares its provenance, access conditions, and licensing — not merely asserting that data exists.
Dataset Resolvability	14%	Whether declared datasets carry resolvable identifiers (accessions, DOIs, repository URLs) that actually resolve to the stated record.
Code Availability & Versioning	14%	Whether the simulation code is publicly archived, versioned, and reachable at a stable location, with an explicit license.
Code-to-Data Traceability	18%	Whether each computational step can be traced back to the specific data source and citation it depends on.
Simulation Derivation Clarity	18%	Whether model parameters, assumptions, and update rules are derived transparently rather than asserted, so the simulation can be re-derived.
Reproducibility Package Quality	8%	Whether a runnable package (environment spec, seeds, instructions) lets an independent reader reproduce the reported results.
Information-Theoretic Rigor	10%	Whether the paper formalizes and quantifies the information content of the system it models. Orthogonal to transparency; non-applicable topics are scored at a neutral baseline (~50).

2.3 Conformance to good data and reproducibility practice

Six of the seven dimensions measure whether a simulation accords with established good practice for data and software. Data Disclosure and Dataset Resolvability test the FAIR ideal that data be findable and accessible through identifiers that genuinely resolve — a declared dataset with no resolvable accession scores poorly even if the prose claims openness. Code Availability & Versioning and Reproducibility Package Quality test whether the software is archived, licensed, versioned, and packaged so an independent reader can run it. Code-to-Data Traceability and Simulation Derivation Clarity test the chain of custody from result back to source: can each computational step and model parameter be traced to the specific data and citation it depends on, rather than asserted? Crucially, these scores are anchored to verified external signals — a resolved DOI, a real license file detected in the repository — and not to the paper's self-description; the framework does not credit unverifiable claims, and does not invent gaps it cannot substantiate.

2.4 Code-to-data traceability

Beyond scoring, BioEval can accept the simulation's source code directly: the model segments the code and maps each segment to the data source and citation it relies on, with a high/medium/low confidence rating. This makes the dependency between a result and its inputs explicit and auditable, which is the operational meaning of reproducibility for a simulation.

3 Information-Theoretic Rigor

The seventh dimension is orthogonal to the other six: it measures scientific-content rigor rather than transparency. Many papers in this domain model communication and coordination systems — pheromone stigmergy in ants and termites, waggle-dance signaling in bees, phase coupling in firefly and Kuramoto

synchronization — where information flow is not a side effect but the phenomenon itself. For such systems, rigor means formalizing that information: the Shannon entropy of agent state or signal distributions, the mutual or transfer entropy that captures directed information flow between agents, and the channel capacity or communication bit rate of the signaling mechanism (bits per pheromone deposit, bits per dance, bits per second), including how these scale with colony size and signal-to-noise.

The guideposts reward papers that actually do this mathematics and penalize papers where communication is central yet treated purely mechanistically. To avoid punishing work for which information theory is simply not applicable — a pure phylogenetics or sequence-alignment pipeline, say — such topics are scored at a neutral midpoint (~50) rather than zero, so non-applicability neither rewards nor punishes. Because the dimension is orthogonal to transparency, a paper can be exemplary in data practice yet score low here, and that contrast is itself the finding this report foregrounds.

4 Evaluation Corpus and Method

We evaluated 14 computational-simulation projects spanning ant, bee, termite, and firefly systems and ant-colony optimization, drawn from the insect-swarm simulation series. Every project was scored under a single rubric version (v0.8.0) so the results are mutually comparable. Scores are assigned by the language model against the fixed guideposts of Section 2, grounded in the external evidence gathered during analysis. In the table below the overall column is recomputed as the exact weighted mean of the seven dimensions, so each row is reproducible from the published weights.

5 Results

5.1 Score summary

Simulation project	Ovr	DD	DR	CA	TR	SC	RP	IT
BeeStack: An Evidence-Typed Scaffold for Whole-Colo…	**64.4**	72	68	71	70	65	52	35
Bee Swarm Simulation	**61.3**	68	72	52	70	55	62	42
The Ant Stack: Reproducible scientific workspace fo…	**42.7**	52	38	62	42	30	45	28
Termite Colony — Unity 3D Multiagent	**42.2**	52	48	55	42	38	28	18
Ant Colony Optimization — React/Canvas	**40.8**	52	55	62	38	22	38	12
Ant Simulation — Pygame Stigmergy	**40.3**	52	58	62	22	28	42	18
Rust Ant Colony Simulation	**40.2**	62	58	52	30	22	38	12
Firefly Synchronization Simulation (JS)	**38.0**	42	48	52	38	30	28	20
Locas Ants — Lua/Love2D Ant Colony	**32.2**	22	35	72	18	18	62	18
Hardware-Accelerated Ant Colony Swarm (C++)	**29.1**	28	42	48	22	15	35	20
ACO Robot Path Planning (Python)	**28.6**	22	30	48	35	22	12	25
Symbiants — Ant Colony Sim (Rust)	**27.0**	22	28	62	15	12	38	25
Ants Sandbox (TypeScript/web)	**24.4**	18	30	62	12	10	35	15
swarm-abc — Artificial Bee Colony Simulation	**20.7**	18	35	32	12	18	8	20

DD Data Disclosure · DR Dataset Resolvability · CA Code Availability · TR Code-to-Data Traceability · SC Simulation Clarity · RP Reproducibility Package · IT Information-Theoretic Rigor. Overall is the weighted mean of all seven dimensions. Scores are rubric v0.8.0 (0–100).

5.2 Observations

Three patterns recur across the corpus. Transparency is uneven: code availability tends to be the strongest dimension — most projects publish a repository — while reproducibility-package quality and code-to-data traceability are consistently the weakest, meaning the code exists but cannot easily be re-run or traced to its inputs. The headline finding is the systematically low Information-Theoretic Rigor across projects whose entire subject is communication and coordination: even strong, transparent simulations rarely quantify the

information their agents exchange. The orthogonality of the rubric makes this visible — high transparency and low information-theoretic rigor coexist in the same papers, marking a concrete and addressable opportunity for the field.

5.3 Per-project synopsis

BeeStack: An Evidence-Typed Scaffold for Whole-Colony Honeybee Simulation — overall 64.4 (IT 35)
BeeStack demonstrates commendable transparency infrastructure: source is publicly archived on Zenodo (DOI: 10. 5281/ zenodo.

Principal gap: GitHub API detects NO license file in the repository itself, contradicting the MIT claim on Zenodo — users cloning from GitHub encounter an ambiguous licensing situation

Bee Swarm Simulation — overall 61.3 (IT 42)
This is a browser-based agent simulation of bee swarm foraging behavior, publicly hosted on GitHub with no external datasets — appropriately evaluated on generative-input disclosure rather than accession numbers. The README is reasonably thorough (~5,400 bytes), documents a citation-backed mode linking waggle-dance parameters to named literature sources (von Frisch 1967, Seeley 1995, Couvillon 2019, Schurch 2021, Shannon 1948, Fisher 1993), and provides a BeeStack JSON schema for external trace replay.

Principal gap: No software license detected in the repository, creating ambiguity about reuse and redistribution rights

The Ant Stack: Reproducible scientific workspace for ant-inspired simulation and complexity energetics — overall 42.7 (IT 28)
The Ant Stack presents a well-structured conceptual framework with a public GitHub repository (ant_stack) containing a passing test suite (676 tests), CLI entrypoints, and a documented artifact policy with SHA-256 checksums and provenance files — meaningful infrastructure for reproducibility. However, the paper is primarily a design specification rather than a completed empirical or simulation study: key contributions (evaluation suites, species presets, open evaluation assets) are described as future deliverables rather than present artifacts, simulation parameters lack pinned versions for core dependencies (MuJoCo, PyBullet, Brian2, Nengo), and the Reproducibility Checklist in Appendix A has no content.

Principal gap: Simulation engine dependencies (MuJoCo, PyBullet, Brian2, Nengo, SpikingJelly) are listed as options without pinned version numbers, making environment reconstruction ambiguous

Termite Colony — Unity 3D Multiagent — overall 42.2 (IT 18)
This is a GitHub-hosted Unity/C# termite colony simulation with a publicly accessible MIT-licensed repository, a basic README, and clearly stated experimental parameters (10 termites, 1000 logs, center spawn at (0,0,0), 1-second direction change interval). However, the project has no versioned release, no pinned commit cited for reported results, no dependency manifest, no CI/tests, a mismatched clone URL in the README, and result images that are referenced but not stably archived.

Principal gap: No tagged release or versioned software release exists (0 releases, 0 tags confirmed via API), so the exact code state used for the reported results cannot be pinpointed

Ant Colony Optimization — React/Canvas — overall 40.8 (IT 12)
This is a GitHub-hosted learning-exercise simulation of ant colony behavior using React and HTML5 canvas, publicly available under the MIT License with a README, package. json dependency manifest, and self-contained assets (tileset, sprite sheet, map JSON).

Principal gap: No versioned releases or Git tags published; no pinned commit cited for reproducibility

Ant Simulation — Pygame Stigmergy — overall 40.3 (IT 18)
This is a publicly available agent-based simulation project with clear installation instructions and an MIT license, but it falls well short of scientific reproducibility standards. The README names five tunable parameters (ANT_COUNT, EVAPORATION_RATE, NEST_GRAVITY_STRENGTH, MAX_PATIENCE, PANIC_DURATION) as the core 'laws of physics' yet reports none of their actual values, leaving no way to reproduce the specific behavior shown in the demo video.

Principal gap: No actual numeric values are reported for any of the five named parameters, making exact reproduction of the demo behavior impossible

Rust Ant Colony Simulation — overall 40.2 (IT 12)
This is a publicly accessible Rust/Bevy ant colony simulation with source code on GitHub, basic README instructions, and a Cargo. toml dependency manifest, but it lacks a license, versioned releases, CI, tests, and any formal parameter documentation.

Principal gap: No software license detected in the repository, preventing legal reuse or redistribution

Firefly Synchronization Simulation (JS) — overall 38.0 (IT 20)
This repository hosts a JavaScript/Arduino firefly synchronization simulation with a public GitHub presence and a live demo URL, providing basic accessibility. However, critical reproducibility infrastructure is absent: there is no license, no versioned release or pinned commit tag, no CI/testing framework, no Dockerfile, and the README is minimal (~42 bytes).

Principal gap: No software license is detected, making reuse, modification, and redistribution legally ambiguous

Locas Ants — Lua/Love2D Ant Colony — overall 32.2 (IT 18)
This repository is a Lua+Löve2D ant colony simulation remake with commendable code availability (MIT license, 6 versioned releases, 161 commits, public GitHub), but scores poorly on scientific rigor dimensions because no simulation parameters, initial conditions, or generative rules are documented in the README or associated materials. There is no linked publication, no parameter sourcing, and no description of the algorithmic choices underlying the ant behavior model, making the simulation scientifically opaque despite being technically runnable.

Principal gap: No simulation parameters (pheromone decay rates, evaporation constants, ant sensing radii, colony size, etc.) are documented anywhere in the README or repository

Hardware-Accelerated Ant Colony Swarm (C++) — overall 29.1 (IT 20)
This repository presents a CUDA/OpenGL-accelerated ant colony swarm simulation for search-and-rescue tasks with an evolutionary parameter optimizer, but falls substantially short of reproducibility standards. The code is publicly accessible on GitHub with build instructions, yet lacks a license, versioned release, dependency manifest with pinned versions, CI, tests, and any documentation of the algorithm's parameters, evolutionary operator configurations, or random seeds.

Principal gap: No software license is present (confirmed via GitHub API: license = none), preventing legal reuse or redistribution

ACO Robot Path Planning (Python) — overall 28.6 (IT 25)
This repository provides a public Python simulation of an ACO-based robot path planning algorithm from Liu et al. (2017), licensed under GPL-3.

Principal gap: README is only ~271 bytes with no parameter documentation, usage instructions, or methodological description

Symbiants — Ant Colony Sim (Rust) — overall 27.0 (IT 25)
This submission is a GitHub-hosted game/simulation project (Symbiants) rather than a conventional scientific paper, which makes many standard reproducibility criteria only partially applicable. The repository is publicly accessible under dual Apache-2.

Principal gap: No tagged releases or pinned commits for reproducibility; the repository points only to the master branch with no version anchor

Ants Sandbox (TypeScript/web) — overall 24.4 (IT 15)
This repository is a browser-based ant colony simulation with publicly available source code under the MIT license, basic build/run instructions, and a dependency manifest (package. json), but it lacks essentially all scientific rigor expected of a reproducible simulation study.

Principal gap: No simulation parameters (ant speed, pheromone evaporation rates, sensing radius, colony size, etc.) are documented anywhere in the README or linked documentation

swarm-abc — Artificial Bee Colony Simulation — overall 20.7 (IT 20)
This repository is a bare-minimum public dump of an Artificial Bee Colony simulation implemented in client-side HTML/JS/CSS with a single commit and virtually no documentation infrastructure. While the code is publicly accessible and the simulation requires no external datasets (which partially mitigates data-disclosure concerns), critical reproducibility elements are entirely absent: no README, no license, no dependency manifest, no versioned release or pinned commit, no tests or CI, no parameter sourcing, and no documentation of the generative rules or initial conditions beyond what is hard-coded in abc.

Principal gap: No README file of any kind is present, making it impossible to understand what the simulation does, how to run it, or what results to expect

6 Discussion

BioEval treats reproducibility as a property that must be demonstrated through resolvable evidence rather than claimed in prose, and it adds a second axis — information-theoretic rigor — that is specific to the communication systems this literature studies. The two axes are complementary: good data practice makes a result checkable, while information-theoretic formalization makes it meaningful for systems whose subject is information. Using a language model as the evaluator is what makes assessment at this breadth tractable; fixed guideposts, evidence grounding, and a versioned rubric are what keep it disciplined.

7 Limitations

Scores are produced by a language model and tend to snap to the discrete tiers defined by the guideposts; feeding real, per-item external signals (resolved accessions, live repository facts) is what differentiates otherwise-similar papers. The rubric is pre-1.0 and still under reviewer validation, so weights and guideposts may change between versions — hence the stamped rubric version on every evaluation. The corpus is domain-specific (insect-swarm and agent-based simulation), and the information-theoretic dimension is calibrated for systems where information flow is central; its neutral-baseline treatment of non-applicable topics is a

deliberate, conservative choice rather than a measurement.

8 Availability and Reproducibility

BioEval is released under the MIT license. The source repository is hosted at github.com/fractastical/bioinformatics-eval, and this Zenodo deposition bundles the present report together with a complete archive of the source code at the corresponding revision. The system is built as a pnpm monorepo (React + Vite frontend, Express 5 API, PostgreSQL with Drizzle ORM, contract-first OpenAPI/Orval codegen) and uses Anthropic Claude through Replit's AI integrations. The evaluation rubric and the software are co-versioned at v0.8.0.

References

[1] Shannon, C. E. (1948). A Mathematical Theory of Communication. Bell System Technical Journal, 27, 379–423, 623–656.

[2] Grassé, P.-P. (1959). La reconstruction du nid et les coordinations interindividuelles: la théorie de la stigmergie. Insectes Sociaux, 6, 41–80.

[3] Dorigo, M., & Stützle, T. (2004). Ant Colony Optimization. MIT Press.

[4] Kuramoto, Y. (1984). Chemical Oscillations, Waves, and Turbulence. Springer.

[5] Schreiber, T. (2000). Measuring Information Transfer. Physical Review Letters, 85(2), 461–464.

[6] Wilkinson, M. D., et al. (2016). The FAIR Guiding Principles for scientific data management and stewardship. Scientific Data, 3, 160018.

[7] Sandve, G. K., et al. (2013). Ten Simple Rules for Reproducible Computational Research. PLoS Computational Biology, 9(10), e1003285.